

A new tree comparison metric - the mixture distance for the mixture trees

Chen-Hui Lin and Justie Su-Tzu Juan

Department of Computer Science and Information Engineering

National Chi Nan University , Nantou, Puli, Taiwan

{s95321545, jsjuan}@ncnu.edu.tw

Abstract

Tree comparison metric is to measure similarity for the computation of trees, and it is an important topic in bioinformation. There are many proposed metrics for the trees comparison, but no one fits to solve the tree comparison between two mixture trees up to now. Mixture trees have more information, like time parameter. It is held in 2006 by Chen and Lindsay and is an important phylogenetic tree. In this paper, the proposed metric is a method to measure the similarity of two mixture trees, and we develop an algorithm for the metric in time $O(n^2)$.

1 Introduction

The phylogenetic trees describe the relationship of a set of species. Using species information to build phylogenetic trees is a popular problem. There are many methods to build trees, like neighbor-joining [10], maximum likelihood [8], and so on. In this topic, to propose a method of building trees needs to do bootstrapping. Different trees may be built by a same data set, even if using the same method [11]. Besides, the comparison of phylogenetic trees is essential when we execute phylogenetic queries on databases of phylogenetic trees [13]. Thus, a metric to measure distance for tree comparison is an important problem.

It is not easy to compare trees. Unlike the comparison of two numbers or points in space [12], there is no obvious or natural way to measure the distance between two trees. Many tree comparison metrics have been proposed, including the partition metric [9], the quartet metric [7], the nearest neighborhood interchange metric [4], the metric from the nodal distance

algorithm [1], and so on. In 2006, Chen and Lindsay proposed a method that can build the mixture tree [3], which is a hierarchical tree that induces a sieve parameter to measure evolutionary time. There are much information on the trees. For example, when the species mutation happens, and where the mutation occurs in the DNA sequences. Figure 1 is an example of a mixture tree. There is no metric to discuss the tree comparison between two mixture trees up to now. In this paper, the proposed metric focuses on the mixture tree comparison.

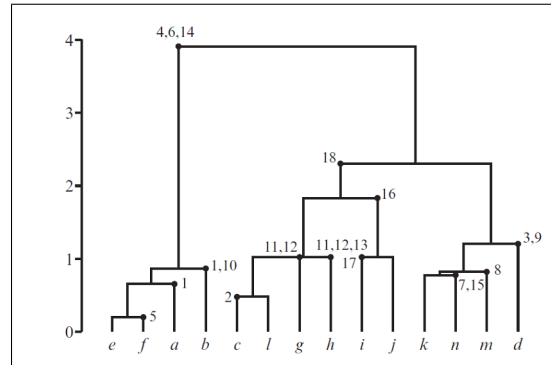


Figure 1: a mixture tree.

There are two related works which are similar our proposed metric. They are path difference metric and the metric from the nodal distance.

Path difference metric [12] - It was one of proposed metrics by Penny and Hendy in 1993. Let $d_{ij}(T)$ denote the number of edge in T in the path joining the leaves labeled i and j , and let $d(T)$ be the associate vector obtained by a fixed ordering of the pairs (i, j) . $d_p(T_1, T_2)$ denotes the Euclidean distance between the

two vector $d(T_1)$ and $d(T_2)$: $\text{Distance}(T_1, T_2) = d_p(T_1, T_2) = \|d(T_1) - d(T_2)\|_2$.

The metric from the nodal distance algorithm [1] - It was proposed by John Bluis and Dong-Guk Shin in 2003. The metric is used to measure the distance of leaves to other leaves for all leaves in a tree. The metric is defined as follows: $\text{Distance}(T_1, T_2) = \sum_{x,y \in V(T_1)=V(T_2)} |D_{T_1}(x, y) - D_{T_2}(x, y)|$. Where $D_{T_i}(x, y)$ denotes the distance of leaf x to leaf y in the tree T_i . The nodal distance algorithm was developed for this metric. The time complexity of this algorithm is $O(n^3)$ for binary unrooted trees.

2 Main Results

In this section, we will give the main result of this study. In Subsection 2.1, the metric will be defined, Subsection 2.2 will develop an algorithm for the proposed metric, and Subsection 2.3 will discuss the complexity of this algorithm.

2.1 The Metric

A *fully resolved tree* is a tree in which every node bifurcates [6], and also is called a *full binary tree*. Throughout this paper, we only discuss the full binary tree, which is a tree $T = (V, E)$ with $|V|$ nodes and n leaves, and each node v_i has either two children or no child. The node without child is called a *leaf*, which is associated with a *species*. Because we discuss a mixture tree, every internal node v_i will be associated with a value P_i , *time parameter*, that denotes the time when a species mutation occurs. We can see the value as the length of v_i to any leaf which is an descender of v_i .

In a tree $T = (V, E)$, let $P_{T_i}(x, y)$ denote the time parameter of the least common ancestor of two leaves x, y in tree T_i . Define a metric, called *mixture distance*, as the sum of the difference of $P_{T_i}(x, y)$ for any two leaves x, y . That is, the mixture distance between two mixture trees T_1, T_2 is defined as $d_m(T_1, T_2) = \sum_{x,y \in V(T_1)=(T_2)} |P_{T_1}(x, y) - P_{T_2}(x, y)|$.

A good definition of distance d will satisfy the following three conditions [5]. First, if tree A is equal to tree B , the distance $d(A, B)$ is zero. Second, the distance of tree A to tree B is equal to the distance of tree B to tree A . That is, $d(A, B) = d(B, A)$. Third, a good definition

must satisfy the triangle inequality. That is, $d(A, B) + d(B, C) \geq d(A, C)$. In our metric, it is not difficult to see that the definition of mixture distance d_m satisfies first and second conditions. For the last condition, it is proved by Theorem 1.

Theorem 1. *The mixture distance d_m satisfied triangle inequality.*

Proof. Let V be the set of all leaves of A , note that V is also equal to the set of all leaves of B and C . Since for all a, b and $c \in \mathbb{R}^+ \cup \{0\}$, $|a - b| + |b - c| \geq |a - c|$. So, for all leaves $x, y \in \{A, B, C\}$, we have $|P_A(x, y) - P_B(x, y)| + |P_B(x, y) - P_C(x, y)| \geq |P_A(x, y) - P_C(x, y)|$. Hence,

$$\begin{aligned} & \sum_{x,y \in V} |P_A(x, y) - P_B(x, y)| \\ & + \sum_{x,y \in V} |P_B(x, y) - P_C(x, y)| \\ & \geq \sum_{x,y \in V} |P_A(x, y) - P_C(x, y)|. \end{aligned}$$

That is, $d_m(A, B) + d_m(B, C) \geq d_m(A, C)$. $\square$

2.2 Algorithm

In [1], the nodal distance algorithm needs to be calculated for every possible combination of two leaves for each of the two trees. There are $2 \cdot \binom{n}{2}$ pair of leaves. If this algorithm executes on our metric, the time complexity is $O(n^3)$. Next we will give an better algorithm as follows. The time complexity will be $O(n^2)$.

Here we only compare the least common ancestor of two leaves in each of two trees. For an internal node in T_1 , we find all pair leaves which the least common ancestor is this node. Then, we find the least common ancestors of those leaves in T_2 , and calculate the distance. In order to implement this approach, similar to [2], we use two colors to label leaves that we found. So *color information* has two parts. One is the amount of red leaves of the subtree, and the other is the amount of green leaves of the subtree. The algorithm is presented as follows.

Input: Two full binary trees T_1 and T_2 .

Output: The mixture distance of T_1 and T_2 .

- **Step 1** Traversal T_1 , and give all internal nodes an order by BFS.
- **Step 2** For each internal node v_{T_1} in T_1 . Do **Step 3** to **Step 7**.
- **Step 3** In T_1 for the subtree which is rooted by v_{T_1} , color leaves of its left subtree by red, and color leaves of its right subtree by green.
- **Step 4** For each internal node v_{T_2} from bottom to top in T_2 . Do **Step 5** to **Step 7**.
- **Step 5** Get *color information* from children, calculate the *color information* of v_{T_2} by adding the associated *color information* of two children of v_{T_2} .
- **Step 6** Count *number* by *color information*: The number is the sum of the amount of red leaves of the left child multiple that of green leaves of the right child and the amount of green leaves of the left child multiple that of red leaves of the right child.
- **Step 7** Count the distance by *number* and multiple the difference of p of v_{T_1} and p of v_{T_2} .
- **Step 8** Output the sum of distance.

Step 5 – Step 7 is described in pseudo code as a recursive procedure *count*() in Figure 2.

2.3 An Example

In T_1 , see Figure 3(a), the leaves of the left subtree of v_1 are {A, B, C, D} that are labelled by red, and the leaves of the left subtree of v_1 are {E, F, G, H} that are labelled by green. The leaves in T_2 are labelled by the same colors. In T_2 , see Figure 3(b), node v_4 gets color information from leaf A (red: 1, green: 0), and leaf B (red: 1, green: 0). In v_4 of T_2 , the distance is $|9 - 1| \times (1 \times 0 + 0 \times 1) = 0$. Node v_2 gets color information from node v_4 (red: 2, green: 0), and node v_5 (red: 0, green: 2). In v_2 of T_2 , the distance is $|9 - 6| \times (2 \times 2 + 0 \times 0) = 12$. Node v_1 gets color information from node v_2 (red: 2, green: 2), and node v_3 (red: 2, green: 2). In v_1 of T_2 , the distance is $|9 - 9| \times (2 \times 2 + 2 \times 2) = 0$. After calculating all nodes of T_2 , choose the next node of T_1 (v_2) for labelling leaves again. Finally, the distance is the sum of the distance of all nodes, it is 47 in this example. In Figure 4, it is the difference of each pair leaves, and in Figure 5, it is the distance of each node in T_1 by the algorithm.

```

Procedure count(root of a subtree in  $T_2$ :  $v$ , associated time parameter of node  $v(i)$  in  $T_1$ :  $v(i).p$ )
{
  IF  $v$  is a leaf THEN
    IF color( $v$ ) = red THEN
      color information = (1, 0)
    ELSEIF color( $v$ ) = green THEN
      color information = (0, 1)
    ELSE
      color information = (0, 0)
  ELSE
     $R$  = count( $v$ .right child,  $v(i).p$ )
     $L$  = count( $v$ .left child,  $v(i).p$ )
    number = ( $L$ .red *  $R$ .green +  $L$ .green *  $R$ .red)
     $D = D + |v(i).p - v.p| * \text{number}$ 
    color information =  $R + L$ 
  ENDIF
  RETURN color information
END
}
    
```

Figure 2: The algorithm.

2.4 Discussion

In this algorithm, we traversal the subtree of each internal node in T_1 in time $O(n)$. Labelling leaves needs time $O(n)$. And for each node of T_1 , to count mixture distance for each pair nodes of T_2 is in time $O(n)$. Hence, the total time complexity is $O(n^2)$, which is better than nodal distance algorithm. Actually, there are $O(n^2)$ $P_{T_i}(x, y)$ in T_i , but there are only $O(n)$ internal nodes in T_i . That is why the proposed algorithm is better. The proposed algorithm can be suitable for ultrametric trees, which is a tree with equal root-to-leaf path lengths for all lineages, and does not be altered. For space complexity, we only need to store those two trees, and the color information of each node. Hence, the space we needed is equal to the constant multiple of the tree size. So, the total space complexity is $O(n)$.

3 Conclusions

The computation of trees needs a metric to measure similarity. In this paper, we presented a metric with time parameter for the mixture trees, and an algorithm for the metric in time $O(n^2)$, which is better than the nodal distance algorithm

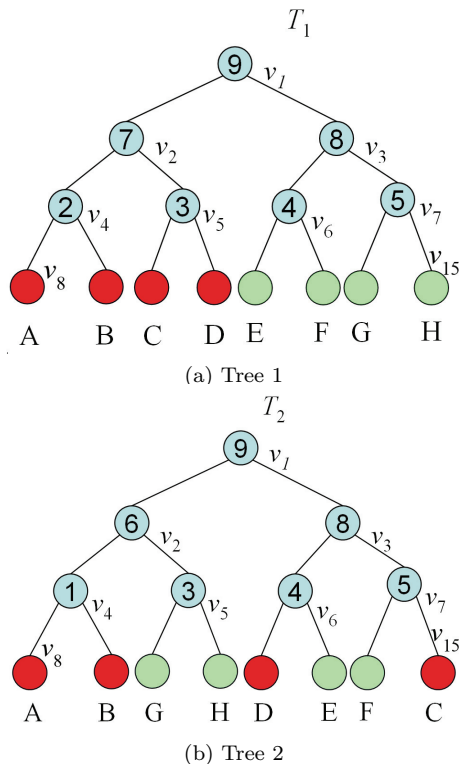


Figure 3: Two trees labelled according to v_1 of T_1 .

with the same metric. Decreasing the time complexity could be a future work. We can also apply the proposed algorithm to the ultrametric trees. Future research directions include designing a faster algorithm, increasing more information in the metric, and extending the algorithm such that it can be suitable for more general trees.

References

- [1] J. Bluis, J. Bluis, and D. Shin, "Nodal distance algorithm: calculating a phylogenetic tree comparison metric," *Proc. Third IEEE Symposium on Bioinformatics and Bioengineering* (D. Shin, ed.), pp. 87–94, 2003.
- [2] G. S. Brodal, R. Fagerberg, and C. N. S. Pedersen, "Computing the quartet distance between evolutionary trees in time $o(n \log n)$," *Algorithmica*, Vol. 38, No. 2, pp. 377–395, 2003.
- [3] S.-C. Chen and B. G. Lindsay, "Building mixture trees from binary sequence data," *Biometrika*, Vol. 93, No. 4, pp. 843–860, 2006.
- [4] B. DasGupta, X. He, T. Jiang, M. Li, J. Tromp, and L. Zhang, "On computing the nearest neighbor interchange distance," *Proceedings of the DIMACS Workshop on Discrete Problems with Medical Applications, DIMACS Series in Discrete Mathematics and Theoretical Computer Science*, American Mathematical Society, Vol. 55, pp. 125–143, 2000.
- [5] J. Felsenstein, *Inferring phylogenies*. Sunderland, MA: Sinauer Associates, 2004.
- [6] M. J. Fortin, M. R. T. Dale, and J. V. Hoef, *Encyclopedia of Environmetrics*, Vol. 4, ch. Spatial analysis in ecology, pp. 2051–2058. John Wiley & Sons, Ltd, 2002.
- [7] C. A. M. George F. Estabrook, F. R. McMorris, "Comparison of undirected phylogenetic trees based on subtrees of four evolutionary units," *Systematic Zoology*, Vol. 34, No. 2, pp. 193–200, 1985.
- [8] M. L. Lesperance and J. D. Kalbfleisch, "An algorithm for computing the nonparametric MLE of a mixing distribution," *Journal of the American Statistical Association*, Vol. 87, No. 417, pp. 120–126, Mar. 1992.
- [9] D. F. Robinson and L. R. Foulds, "Comparison of phylogenetic trees," *Mathematical Biosciences*, Vol. 53, No. 1-2, pp. 131–147, February 1981.
- [10] N. Saitou and M. Nei, "The neighbor-joining method: a new method for reconstructing phylogenetic trees," *Mol Biol Evol*, Vol. 4, No. 4, pp. 406–425, Jul 1987.
- [11] M. A. Steel, "The maximum likelihood point for a phylogenetic tree is not unique," *Systematic Biology*, Vol. 43, pp. 560–564, 1994.
- [12] M. A. Steel and D. Penny, "Distributions of tree comparison metrics-some new results," *Systematic Biology*, Vol. 42, No. 2, pp. 126–141, 1993.
- [13] G. Valiente, "A fast algorithmic technique for comparing large phylogenetic trees," *SPIRE*, pp. 370–375, 2005.

AB	$ T_1(v_4) - T_2(v_4) = 2 - 1 = 1$
AC	$ T_1(v_2) - T_2(v_1) = 7 - 9 = 2$
AD	$ T_1(v_2) - T_2(v_1) = 7 - 9 = 2$
AE	$ T_1(v_1) - T_2(v_1) = 9 - 9 = 0$
AF	$ T_1(v_1) - T_2(v_1) = 9 - 9 = 0$
AG	$ T_1(v_1) - T_2(v_2) = 9 - 6 = 3$
AH	$ T_1(v_1) - T_2(v_2) = 9 - 6 = 3$
BC	$ T_1(v_2) - T_2(v_1) = 7 - 9 = 2$
BD	$ T_1(v_2) - T_2(v_1) = 7 - 9 = 2$
BE	$ T_1(v_1) - T_2(v_1) = 9 - 9 = 0$
BF	$ T_1(v_1) - T_2(v_1) = 9 - 9 = 0$
BG	$ T_1(v_1) - T_2(v_2) = 9 - 6 = 3$
BH	$ T_1(v_1) - T_2(v_2) = 9 - 6 = 3$
CD	$ T_1(v_5) - T_2(v_3) = 3 - 8 = 5$
CE	$ T_1(v_1) - T_2(v_3) = 9 - 8 = 1$
CF	$ T_1(v_1) - T_2(v_7) = 9 - 5 = 4$
CG	$ T_1(v_1) - T_2(v_1) = 9 - 9 = 0$
CH	$ T_1(v_1) - T_2(v_1) = 9 - 9 = 0$
DE	$ T_1(v_1) - T_2(v_6) = 9 - 4 = 5$
DF	$ T_1(v_1) - T_2(v_3) = 9 - 8 = 1$
DG	$ T_1(v_1) - T_2(v_1) = 9 - 9 = 0$
DH	$ T_1(v_1) - T_2(v_1) = 9 - 9 = 0$
EF	$ T_1(v_6) - T_2(v_3) = 4 - 8 = 4$
EG	$ T_1(v_3) - T_2(v_1) = 8 - 9 = 1$
EH	$ T_1(v_3) - T_2(v_1) = 8 - 9 = 1$
FG	$ T_1(v_3) - T_2(v_1) = 8 - 9 = 1$
FH	$ T_1(v_3) - T_2(v_1) = 8 - 9 = 1$
GH	$ T_1(v_7) - T_2(v_5) = 5 - 3 = 2$
Total 47	

Figure 4: Count 1

$T_1(v_1) - T_2(v_6) = 9 - 4 = 5$	
$T_1(v_1) - T_2(v_7) = 9 - 5 = 4$	
$T_1(v_1) - T_2(v_2) = 9 - 6 * 2 * 2 = 12$	
$T_1(v_1) - T_2(v_3) = 9 - 8 * 2 = 2$	
$T_1(v_1) - T_2(v_1) = 9 - 9 * (2 * 2 + 2 * 2) = 0$	
$T_1(v_2) - T_2(v_1) = 7 - 9 * 2 * 2 = 8$	
$T_1(v_3) - T_2(v_1) = 7 - 9 * 2 * 2 = 4$	
$T_1(v_4) - T_2(v_4) = 2 - 1 = 1$	
$T_1(v_5) - T_2(v_3) = 3 - 8 = 5$	
$T_1(v_6) - T_2(v_3) = 4 - 8 = 4$	
$T_1(v_7) - T_2(v_5) = 5 - 3 = 2$	
Total 47	

Figure 5: Count 2